

TECH

API / GRAPHQL / NETWORKING

API / GRAPHQL / NETWORKING

API / GRAPHQL / NETWORKING

30 ВОПРОСОВ 7 ДНЕЙ КОНСПЕКТ ЛЕКЦИЙ

ИСТОЧНИК graphql.org / [tanstack](https://tanstack.com) / общая практика

20 ВОПРОСОВ 7 ДНЕЙ

ДЕНЬ 1 СРЕДНИЙ	REST vs GRAPHQL Q1 · Q2 · Q3 · Q4 · Q5	OVER/UNDERFETCH	REST vs GRAPHQL	OVER/UNDERFETCH
ДЕНЬ 2 СРЕДНИЙ	GQL TYPES Q6 · Q7 · Q8	ERRORS	CACHE	GQL TYPES ERRORS CACHE
ДЕНЬ 3 СРЕДНИЙ	PAGINATION IGN Q9 · Q10 · Q11 · Q12	OPTIMISTIC	CLIENT DES	PAGINATION IGN OPTIMISTIC CLIENT DES
ДЕНЬ 4 СРЕДНИЙ	CANCELLATION DEMPOTENCY Q13 · Q14 · Q15 · Q16	RETRY	NO-RETRY	I CANCELLATION DEMPOTENCY RETRY NO-RETRY I
ДЕНЬ 5 ТЯЖЁЛЫЙ	RATE LIMIT Q17 · Q18 · Q19	401/403	TOKEN REFRESH	RATE LIMIT 401/403 TOKEN REFRESH
ДЕНЬ 6 ТЯЖЁЛЫЙ	WATERFALL Q20 · Q21 · Q22 · Q23 · Q24 · Q25	PREFETCH	DUPS	BFF WATERFALL PREFETCH DUPS BFF
ДЕНЬ 7 СРЕДНИЙ	CONTRACT ION PARTIAL Q26 · Q27 · Q28 · Q29 · Q30	BACKWARDS COMPAT SLOW	MIGRAT	CONTRACT ION PARTIAL SLOW MIGRAT
РИТМ 30-60 МИН ТЕОРИЯ 15-30 МИН КОД В КОНСОЛИ 15 МИН ОТВЕТЫ ВСЛУХ				
ДЕНЬ 7 — БЕЗ НОВОЙ ТЕОРИИ ТОЛЬКО ПРОГОН ПО 20 ВОПРОСАМ				

ДЕНЬ 1 НАГРУЗКА СРЕДНИЙ 5 ВОПРОСОВ

REST vs GRAPHQL OVER/UNDERFETCH

ЧТО УЧИМ В ЭТОТ ДЕНЬ

REST vs GRAPHQL OVER/UNDERFETCH

ВОПРОСЫ ДНЯ

- Q01 REST или GraphQL – что выбрать?
- Q02 Какие плюсы GraphQL для frontend?
- Q03 Какие минусы GraphQL?
- Q04 Что такое overfetching?
- Q05 Что такое underfetching?

REST или GraphQL – что выбрать?

ОПИСАНИЕ

REST – простота, кэшируемость, многолетний опыт. GraphQL – клиент-driven запросы, точные данные, типизация. Выбор зависит от: размер команды, сложность UI, кэширование, существующий стек.

КАК РАБОТАЕТ

- 01 REST: simple, cacheable, mature.
- 02 GraphQL: precise queries, single endpoint, typed.
- 03 REST + JSON Schema часто достаточно.
- 04 GraphQL ценен на сложных UI с разными views.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Простой CRUD → REST.
- 02 Сложные UI с varying data → GraphQL.
- 03 Existing REST → не мигрировать без причины.

ПОДВОДНЫЕ КАМНИ

- 01 GraphQL без caching стратегии – N+1.
- 02 REST с кучей endpoints – overhead для FE.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое hybrid REST/GraphQL?
- 02 Когда tRPC лучше обоих?

ПОЛНАЯ СТАТЬЯ

<https://graphql.org/learn/>

ВОПРОС Q02

Какие плюсы GraphQL для frontend?

ОПИСАНИЕ

Точно те данные, что нужны (нет over/underfetch). Single endpoint. Типизация (codegen TS-types из schema). Frontend-driven – можно менять запросы без backend.

КАК РАБОТАЕТ

- 01 No over/underfetch.
- 02 Single endpoint.
- 03 Codegen TS types.
- 04 Frontend-driven queries.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Сложные UI с разными views.
- 02 Multi-team app.
- 03 Mobile-friendly bandwidth.

ПОДВОДНЫЕ КАМНИ

- 01 Кэш сложнее REST.
- 02 N+1 на бэке.
- 03 Overhead schema management.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое query complexity?
- 02 Как фронт меняет запросы без релиза BE?

ПОЛНАЯ СТАТЬЯ

<https://graphql.org/learn/best-practices/>

Какие минусы GraphQL?

ОПИСАНИЕ

Сложность кэширования, N+1 проблема на бэке, необходимость server-side complexity limits, тяжёлый client (Apollo / Relay), документация и onboarding, security (introspection in prod).

КАК РАБОТАЕТ

- 01 Caching сложнее REST.
- 02 N+1 в resolvers.
- 03 Heavy clients.
- 04 Need rate-limiting by complexity.
- 05 Introspection in prod = security.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 При выборе stack.
- 02 Аудит существующего GraphQL.

ПОДВОДНЫЕ КАМНИ

- 01 Без DataLoader – N+1 storm.
- 02 Без depth-limit – DoS-attack.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое DataLoader?
- 02 Что такое query depth limit?

ПОЛНАЯ СТАТЬЯ

<https://graphql.org/learn/best-practices/>

ВОПРОС Q04

Что такое overfetching?

ОПИСАНИЕ

Получение от API больше данных, чем нужно UI. Типично для REST: эндпоинт возвращает всю User-модель, а UI показывает только name + avatar. Решения: GraphQL (precise queries), REST с sparse-fields (?fields=...), BFF.

КАК РАБОТАЕТ

- 01 REST: GET /users/1 → all fields.
- 02 UI uses 2 of 20 fields.
- 03 Bandwidth waste, slow parse.
- 04 Solutions: GraphQL, sparse fields, BFF.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Mobile / slow networks.
- 02 Large entities.

ПОДВОДНЫЕ КАМНИ

- 01 Игнор overfetching → bandwidth bloat.
- 02 Sparse fields без cache key – мисс.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое sparse fieldsets?
- 02 Чем BFF помогает?

ПОЛНАЯ СТАТЬЯ

<https://jsonapi.org/format/#fetching-sparse-fieldsets>

Что такое underfetching?

ОПИСАНИЕ

Получение недостаточно данных одним запросом, нужны последовательные roundtrips (waterfall). Типично REST: GET /user → GET /user/posts → GET /user/posts/N/comments. GraphQL решает одним запросом.

КАК РАБОТАЕТ

- 01 REST: GET /a → GET /b → GET /c.
- 02 Waterfall latency.
- 03 Solutions: nested GraphQL, batch endpoints, prefetch.
- 04 Particularly bad on mobile.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Detail-страницы.
- 02 Cascading data.

ПОДВОДНЫЕ КАМНИ

- 01 Mobile-3G делает waterfall катастрофой.
- 02 Без parallel fetching – $N \times \text{latency}$.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое waterfall?
- 02 Чем prefetch помогает?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/optimize-fcp>

ДЕНЬ 2 НАГРУЗКА СРЕДНИЙ 3 ВОПРОСА

GQL TYPES**ERRORS****CACHE**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

GQL TYPES

ERRORS

CACHE

ВОПРОСЫ ДНЯ

Q06 Как типизировать GraphQL queries?

Q07 Как обрабатывать GraphQL errors?

Q08 Как кешировать GraphQL data?

Как типизировать GraphQL queries?

ОПИСАНИЕ

Codegen из schema: graphql-codegen генерирует TS-types для каждого operation. Use them в Apollo / urql / @tanstack/react-query. Типы синхронизированы с backend, breaking changes ловятся в CI.

КАК РАБОТАЕТ

- 01 graphql-codegen + plugins.
- 02 Generated TS types per operation.
- 03 Type-safe Apollo hooks.
- 04 CI fails on breaking schema.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 GraphQL-проекты.
- 02 Multi-team.

ПОДВОДНЫЕ КАМНИ

- 01 Manual types – рассинхрон.
- 02 Codegen без CI – устаревший код.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое client-preset?
- 02 Чем gql-tag-fragment-types помогает?

ПОЛНАЯ СТАТЬЯ

<https://the-guild.dev/graphql/codegen>

ВОПРОС Q07

Как обрабатывать GraphQL errors?

ОПИСАНИЕ

GraphQL имеет 2 типа: protocol errors (500, network) и application errors (response.errors). Best practice – typed errors через union в schema (Result<Data, Error>). Apollo / urql имеют onError callbacks.

КАК РАБОТАЕТ

- 01 data + errors в response.
- 02 Typed errors через union.
- 03 onError-handlers.
- 04 Network vs application errors.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой GraphQL клиент.

ПОДВОДНЫЕ КАМНИ

- 01 Игнор errors при наличии data – partial fail.
- 02 Mix protocol + application errors.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое typed errors?
- 02 Чем Apollo errorLink полезен?

ПОЛНАЯ СТАТЬЯ

<https://the-guild.dev/blog/graphql-error-handling>

Как кешировать GraphQL data?

ОПИСАНИЕ

Apollo / urql / Relay используют normalized cache (по __typename + id). Каждый объект – один раз в кеше, все queries используют ту же ссылку. Invalidation через refetch или manual update.

КАК РАБОТАЕТ

- 01 Normalized cache by __typename+id.
- 02 Auto dedup queries.
- 03 Cache update via writeQuery / writeFragment.
- 04 Refetch / invalidate on mutation.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 SPA с many shared entities.
- 02 Real-time UIs.

ПОДВОДНЫЕ КАМНИ

- 01 Без id поля – cache не работает.
- 02 Manual updates легко расходятся.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое normalized cache?
- 02 Чем Apollo от Relay отличается?

ПОЛНАЯ СТАТЬЯ

<https://www.apollographql.com/docs/react/caching/overview>

ДЕНЬ 3 НАГРУЗКА СРЕДНИЙ 4 ВОПРОСА

PAGINATION

OPTIMISTIC

CLIENT DESIGN

ЧТО УЧИМ В ЭТОТ ДЕНЬ

PAGINATION OPTIMISTIC CLIENT DESIGN

ВОПРОСЫ ДНЯ

- Q09 Как работать с pagination?
- Q10 Cursor pagination или offset pagination?
- Q11 Как обрабатывать optimistic mutations?
- Q12 Как проектировать API client?

Как работать с pagination?

ОПИСАНИЕ

Два подхода: offset (page=1, limit=20) – простой, но performance падает на больших данных и race condition при insert. Cursor (next/prev cursor) – стабильный для real-time, сложнее реализация.

КАК РАБОТАЕТ

- 01 Offset: simple, page-based.
- 02 Cursor: stable for real-time.
- 03 Infinite scroll = cursor.
- 04 Hybrid: cursor для realtime, offset для simple.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Lists, feeds, messages.

ПОДВОДНЫЕ КАМНИ

- 01 Offset duplicates при insert.
- 02 Cursor сложнее для UI 'go to page 5'.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое keyset pagination?
- 02 Чем cursor от offset лучше?

ПОЛНАЯ СТАТЬЯ

<https://relay.dev/graphql/connections.htm>

ВОПРОС Q10

Cursor pagination или offset pagination?

ОПИСАНИЕ

Cursor – для real-time / infinite scroll / больших коллекций (offset деградирует).
Offset – для simple use-cases с фиксированным dataset, где нужно 'page 5 of 10'. Часто гибрид.

КАК РАБОТАЕТ

- 01 Cursor: insert-stable, fast on large.
- 02 Offset: arbitrary jump.
- 03 Cursor – opaque ID for client.
- 04 Cursor по умолчанию для GraphQL connections.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Real-time feeds → cursor.
- 02 Admin tables → offset OK.

ПОДВОДНЫЕ КАМНИ

- 01 Cursor + jump-to-page → не работает.
- 02 Offset на 1M+ rows → slow.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое Relay-style cursor?
- 02 Чем opaque cursor лучше id?

ПОЛНАЯ СТАТЬЯ

<https://relay.dev/graphql/connections.htm>

Как обрабатывать optimistic mutations?

ОПИСАНИЕ

Apply optimistic update в cache → отправить mutation → on success: invalidate / refetch; on error: rollback к snapshot. Apollo + Relay имеют built-in. react-query через onMutate/onError.

КАК РАБОТАЕТ

- 01 onMutate: snapshot + apply.
- 02 onError: rollback.
- 03 onSettled: refetch.
- 04 cancelQueries to prevent race.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Likes, favorites.
- 02 Inline edit.

ПОДВОДНЫЕ КАМНИ

- 01 Без cancelQueries – race с refetch.
- 02 Snapshot потерян → нечего откатывать.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое cancelQueries?
- 02 Чем optimistic полезен на mobile?

ПОЛНАЯ СТАТЬЯ

<https://tanstack.com/query/latest/docs/framework/react/guides/optimistic-updates>

ВОПРОС Q12

Как проектировать API client?

ОПИСАНИЕ

Слои: low-level fetch wrapper (auth, retry, timeout) → resource-specific clients (userApi, postApi) → hooks (useUser). Type-safety через схему, runtime валидация через Zod, обёртки над response.

КАК РАБОТАЕТ

- 01 Layer 1: fetch wrapper (auth, retry).
- 02 Layer 2: resource clients.
- 03 Layer 3: hooks (useUser).
- 04 Zod / Schema for runtime validation.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой fetch к BE.
- 02 Replace ad-hoc fetch.

ПОДВОДНЫЕ КАМНИ

- 01 Inline fetch повсюду – невозможно тестить.
- 02 Без types – runtime баги.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое axios interceptors?
- 02 Чем openapi-typescript полезен?

ПОЛНАЯ СТАТЬЯ

<https://github.com/drwpow/openapi-typescript>

CANCELLATION**RETRY****NO-RETRY****IDEMPOTENCY**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

CANCELLATION

RETRY

NO-RETRY

IDEMPOTENCY

ВОПРОСЫ ДНЯ

Q13 Как проектировать request cancellation?

Q14 Как проектировать retry policy?

Q15 Какие запросы нельзя retry?

Q16 Как проектировать idempotency?

ВОПРОС Q13

Как проектировать request cancellation?

ОПИСАНИЕ

AbortController в каждый fetch. signal в options. Cleanup в useEffect / mutation. react-query / SWR делают автоматически. AbortError ловить отдельно от network errors.

КАК РАБОТАЕТ

- 01 AbortController + signal.
- 02 cleanup → ctrl.abort().
- 03 react-query auto.
- 04 Catch AbortError separately.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Search-typeahead.
- 02 useEffect с props deps.
- 03 Long-running mutations.

ПОДВОДНЫЕ КАМНИ

- 01 Без abort – race condition.
- 02 AbortError as network error → false alert.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое AbortSignal.timeout?
- 02 Как комбинировать сигналы?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/API/AbortController>

Как проектировать retry policy?

ОПИСАНИЕ

Exponential backoff (200, 400, 800ms) + jitter. Max attempts (3-5). Только idempotent методы (GET, idempotent POST с key). Учитывать ошибку (5xx + network – yes; 4xx кроме 429 – no).

КАК РАБОТАЕТ

- 01 Exponential backoff + jitter.
- 02 maxAttempts.
- 03 Idempotent only.
- 04 429 → respect Retry-After.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Network-flaky environments.
- 02 react-query default retry.

ПОДВОДНЫЕ КАМНИ

- 01 Retry на non-idempotent → дубль.
- 02 Без backoff → thundering herd.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое Retry-After?
- 02 Чем jitter помогает?

ПОЛНАЯ СТАТЬЯ

https://en.wikipedia.org/wiki/Exponential_backoff

ВОПРОС Q15

Какие запросы нельзя retry?

ОПИСАНИЕ

Non-idempotent без idempotency-key: POST create без key – может создать дубль. PATCH без version-check – конфликт. DELETE – обычно идиempotent. 4xx (кроме 429) – повторение бесполезно.

КАК РАБОТАЕТ

- 01 POST без idempotency-key.
- 02 PATCH без version.
- 03 Auth-failures (401).
- 04 Validation errors (400).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Code review.
- 02 API client config.

ПОДВОДНЫЕ КАМНИ

- 01 Retry create-order → дубль ордера.
- 02 Retry login после 401 → block account.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое idempotency-key?
- 02 Чем DELETE идиempotent?

ПОЛНАЯ СТАТЬЯ

https://stripe.com/docs/api/idempotent_requests

Как проектировать idempotency?

ОПИСАНИЕ

Idempotency-Key header в POST. Generate UUID при mount формы, retry uses same key. Server: store key+result, при повторе – возвращает cached. Очищать через TTL (24h).

КАК РАБОТАЕТ

- 01 Idempotency-Key UUID.
- 02 Generated once per form mount.
- 03 Server stores key→result.
- 04 TTL для cleanup.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Payments.
- 02 Order create.
- 03 Money transfer.

ПОДВОДНЫЕ КАМНИ

- 01 Новый key per retry → дубль.
- 02 Without server support – meaningless.

МОГУТ ДОСПРОСИТЬ

- 01 Как генерировать ключ?
- 02 Чем server обрабатывает?

ПОЛНАЯ СТАТЬЯ

https://stripe.com/docs/api/idempotent_requests

ДЕНЬ 5 НАГРУЗКА ТЯЖЁЛЫЙ 3 ВОПРОСА

RATE LIMIT 401/403 TOKEN REFRESH

ЧТО УЧИМ В ЭТОТ ДЕНЬ

RATE LIMIT 401/403 TOKEN REFRESH

ВОПРОСЫ ДНЯ

Q17 Как обрабатывать rate limits?

Q18 Как обрабатывать 401/403?

Q19 Как обновлять token?

Как обрабатывать rate limits?

ОПИСАНИЕ

Server returns 429 + Retry-After header. Client: уважать Retry-After (wait), exponential backoff если нет header, queue requests, показать UI 'too many requests, retry in N'.

КАК РАБОТАЕТ

- 01 429 → respect Retry-After.
- 02 Backoff if no header.
- 03 Queue retries.
- 04 Show user-friendly message.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Public APIs.
- 02 Mobile apps.
- 03 Bursty traffic.

ПОДВОДНЫЕ КАМНИ

- 01 Ignore Retry-After → permanent throttling.
- 02 Spam retries при 429 → ban.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое token bucket?
- 02 Чем sliding window отличается?

ПОЛНАЯ СТАТЬЯ

<https://datatracker.ietf.org/doc/html/rfc6585#section-4>

ВОПРОС Q18

Как обрабатывать 401/403?

ОПИСАНИЕ

401 – не авторизован: попытка refresh token, если fail – logout + redirect /login. 403 – auth есть, но нет permission: показать 'access denied' + контакт админа. Не путать.

КАК РАБОТАЕТ

- 01 401 → refresh token → retry.
- 02 403 → access denied UI.
- 03 Singleton refresh-promise (см. debugging Q20).
- 04 Logout state cleanup.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 JWT-based auth.
- 02 RBAC apps.

ПОДВОДНЫЕ КАМНИ

- 01 401 как 403 – false logout.
- 02 Race на параллельных 401 → multiple refreshes.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое axios interceptor?
- 02 Где хранить refresh-token?

ПОЛНАЯ СТАТЬЯ

<https://datatracker.ietf.org/doc/html/rfc7235>

Как обновлять token?

ОПИСАНИЕ

Refresh-token flow: при 401 → POST /refresh с refresh-token → save new access-token → retry original. Singleton refresh-promise для concurrency. Refresh-token в httpOnly cookie или secure storage.

КАК РАБОТАЕТ

- 01 POST /refresh с refresh-token.
- 02 Update access-token.
- 03 Retry original request.
- 04 Singleton refresh-promise.
- 05 httpOnly cookie для refresh-token.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 JWT-auth.
- 02 SPA с long-living sessions.

ПОДВОДНЫЕ КАМНИ

- 01 Refresh в localStorage = XSS-vulnerable.
- 02 Без singleton → race condition.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое silent refresh?
- 02 Чем httpOnly cookie защищает?

ПОЛНАЯ СТАТЬЯ

<https://datatracker.ietf.org/doc/html/rfc6749#section-6>

ДЕНЬ 6

НАГРУЗКА

ТЯЖЁЛЫЙ

6 ВОПРОСОВ

WATERFALL**PREFETCH****DUPS****BFF**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

WATERFALL

PREFETCH

DUPS

BFF

ВОПРОСЫ ДНЯ

- Q20 Как избежать request waterfall?
- Q21 Как prefetch data?
- Q22 Как избежать duplicate requests?
- Q23 Как проектировать BFF?
- Q24 Когда нужен BFF?
- Q25 Какие риски у BFF?

Как избежать request waterfall?

ОПИСАНИЕ

Параллелить независимые запросы (Promise.all, react-query parallel queries).
Server-side: batch endpoints (BFF, GraphQL). Prefetch на route-change. Suspense с разделёнными boundaries для streaming.

КАК РАБОТАЕТ

- 01 Promise.all для независимых.
- 02 useQueries в react-query.
- 03 BFF / GraphQL для batching.
- 04 Prefetch on hover/route.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Detail-страницы.
- 02 Dashboards.

ПОДВОДНЫЕ КАМНИ

- 01 Sequential await вместо Promise.all.
- 02 Без BFF — N клиентских запросов.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое prefetching?
- 02 Чем BFF полезен?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/optimize-fcp>

ВОПРОС Q 2 1

Как prefetch data?

ОПИСАНИЕ

На hover ссылки → начать загрузку (link prefetch / queryClient.prefetchQuery). На route-change → загрузить вероятный следующий screen. На viewport (intersection) → загрузить near-screen контент.

КАК РАБОТАЕТ

- 01 onMouseEnter → prefetch.
- 02 Route-loader (Next/Remix).
- 03 prefetchQuery в react-query.
- 04 <link rel="prefetch"> для статики.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Navigation links.
- 02 Common next-screens.
- 03 Image-galleries.

ПОДВОДНЫЕ КАМНИ

- 01 Prefetch всё → bandwidth waste.
- 02 Prefetch на mobile dataplan – дорого.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое link prefetch?
- 02 Чем queryClient.prefetchQuery полезен?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/link-prefetch>

Как избежать duplicate requests?

ОПИСАНИЕ

react-query / Apollo делают dedup automatically (через queryKey / __typename+id). Для custom client – своя dedup (Map<key, Promise>). Cache TTL для предотвращения lookup-storms.

КАК РАБОТАЕТ

- 01 queryKey-based dedup.
- 02 __typename+id GraphQL.
- 03 Map<key, in-flight Promise>.
- 04 Cache TTL.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Mount нескольких компонентов с одним запросом.
- 02 Race в обработчиках.

ПОДВОДНЫЕ КАМНИ

- 01 Без dedup → N одинаковых запросов.
- 02 Cache too long → stale.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое staleTime?
- 02 Чем dedup от cache отличается?

ПОЛНАЯ СТАТЬЯ

<https://tanstack.com/query/latest/docs/framework/react/guides/important-defaults>

В О П Р О С Q 2 3

Как проектировать BFF?

ОПИСАНИЕ

BFF (Backend-For-Frontend) – middleware, аккумулирующий несколько backend-сервисов в один frontend-friendly API. Решает: aggregate, transform, cache, secure (hide internal API). Свой per-frontend.

КАК РАБОТАЕТ

- 01 Aggregate множественных backend.
- 02 Transform DTO для FE.
- 03 Hide internal endpoints.
- 04 Cache server-side.
- 05 Per-frontend (web/mobile).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Microservices BE.
- 02 Multi-frontend orgs.
- 03 Mobile + Web с разными нуждами.

ПОДВОДНЫЕ КАМНИ

- 01 BFF становится monolith.
- 02 Without versioning → rigid coupling.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое API gateway vs BFF?
- 02 Чем GraphQL похож на BFF?

ПОЛНАЯ СТАТЬЯ

<https://samnewman.io/patterns/architectural/bff/>

Когда нужен BFF?

ОПИСАНИЕ

Когда: backend – microservices с разными форматами, frontend нуждается в aggregate, требуется hide internal API, разные frontends (web/mobile/iot) с разными нуждами.

КАК РАБОТАЕТ

- 01 Many micro-services on BE.
- 02 Aggregation needed.
- 03 Hide internal API.
- 04 Multi-frontend org.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Enterprise SaaS.
- 02 Multi-platform products.

ПОДВОДНЫЕ КАМНИ

- 01 Single FE + monolith BE → BFF лишний.
- 02 Шум поверх простого API.

МОГУТ ДОСПРОСИТЬ

- 01 Чем BFF от gateway отличается?
- 02 Кто owner BFF?

ПОЛНАЯ СТАТЬЯ

<https://samnewman.io/patterns/architectural/bff/>

ВОПРОС Q25

Какие риски у BFF?

ОПИСАНИЕ

Дополнительный сервис для maintain (deploy, monitor, scale). Может стать bottleneck (single point of failure). Может превратиться в monolith. Может дублировать логику с backend.

КАК РАБОТАЕТ

- 01 Operational overhead.
- 02 SPOF risk.
- 03 BFF-monolith tendency.
- 04 Logic duplication with BE.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 BFF adoption.
- 02 Architecture review.

ПОДВОДНЫЕ КАМНИ

- 01 BFF без monitoring → blind.
- 02 BFF без auth → security.

МОГУТ ДОСПРОСИТЬ

- 01 Как scaling BFF?
- 02 Чем отличается от gateway?

ПОЛНАЯ СТАТЬЯ

<https://martinfowler.com/articles/gateway-pattern.html>

CONTRACT**BACKWARDS COMPAT****MIGRATION****PARTIAL****S**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

CONTRACT

BACKWARDS COMPAT

MIGRATION

PARTIAL

SLOW

ВОПРОСЫ ДНЯ

Q26 Как делать contract testing?

Q27 Как обрабатывать backwards compatibility?

Q28 Как мигрировать API contract?

Q29 Как работать с partial response?

Q30 Как обрабатывать slow API?

ВОПРОС Q26

Как делать contract testing?

ОПИСАНИЕ

Pact / Schemathesis: фронт и бэк независимо тестируют свой контракт. FE: моки based on contract. BE: real-server тесты против contract. Проверяет, что обе стороны соблюдают.

КАК РАБОТАЕТ

- 01 Pact: consumer-driven contract.
- 02 FE: mock from contract.
- 03 BE: provider verify.
- 04 CI runs both.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Multi-team org.
- 02 Microservices.
- 03 Backend-frontend split.

ПОДВОДНЫЕ КАМНИ

- 01 Без CI – контракт не enforced.
- 02 Mock-only тесты – false confidence.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое consumer-driven?
- 02 Чем OpenAPI-contract отличается?

ПОЛНАЯ СТАТЬЯ

<https://docs.pact.io/>

Как обрабатывать backwards compatibility?

ОПИСАНИЕ

Add new fields as optional, deprecate before remove (warning header / response field).
Version API (v1, v2 routes), maintain старые versions некоторое время. Tools:
openapi-diff для CI.

КАК РАБОТАЕТ

- 01 New fields = optional.
- 02 Deprecation warning перед removal.
- 03 Version routes /v1 /v2.
- 04 openapi-diff в CI.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Public APIs.
- 02 Long-lived clients.
- 03 Mobile apps.

ПОДВОДНЫЕ КАМНИ

- 01 Breaking change без version → broken clients.
- 02 Versioning forever → maintenance hell.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое API versioning strategy?
- 02 Сколько maintain старых versions?

ПОЛНАЯ СТАТЬЯ

<https://www.openapis.org/>

ВОПРОС Q28

Как мигрировать API contract?

ОПИСАНИЕ

Strangler-pattern: новые fields/endpoints add, старые deprecate. Параллельный rollout (старый + новый). Notify clients (changelog, deprecation warnings). Sunset date с monitoring usage.

КАК РАБОТАЕТ

- 01 Strangler-fig pattern.
- 02 Параллельный rollout.
- 03 Deprecation warnings.
- 04 Sunset с monitoring.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Большие API migrations.
- 02 Breaking changes.

ПОДВОДНЫЕ КАМНИ

- 01 Forced cut-off без grace period.
- 02 Без monitoring – отключаем live clients.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое strangler-pattern?
- 02 Как monitor usage старого API?

ПОЛНАЯ СТАТЬЯ

<https://martinfowler.com/bliki/StranglerFigApplication.html>

Как работать с partial response?

ОПИСАНИЕ

Server returns partial data (some fields fail) – обычно в GraphQL (errors + data). Frontend: показать что есть, indicate failed parts. Не падать целиком, fallback per field.

КАК РАБОТАЕТ

- 01 GraphQL: data + errors.
- 02 REST: 207 Multi-Status.
- 03 Show what's available.
- 04 Indicate failed parts.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Dashboards с N виджетов.
- 02 GraphQL apps.

ПОДВОДНЫЕ КАМНИ

- 01 Fail entire UI на partial fail – UX.
- 02 Игнор errors – silent data loss.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое 207 Multi-Status?
- 02 Чем GraphQL partial response полезен?

ПОЛНАЯ СТАТЬЯ

<https://datatracker.ietf.org/doc/html/rfc4918>

ВОПРОС Q30

Как обрабатывать slow API?

ОПИСАНИЕ

Timeout (`AbortSignal.timeout`). Loading-state с indicator. Skeleton UI вместо spinner. Cache previous data. Show 'still loading...' после 5s. Suggest action (cancel, refresh).

КАК РАБОТАЕТ

- 01 `AbortSignal.timeout(ms)`.
- 02 Skeleton UI.
- 03 `placeholderData` в `react-query`.
- 04 Long-loading message after threshold.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Mobile / slow networks.
- 02 Heavy aggregations.
- 03 Third-party APIs.

ПОДВОДНЫЕ КАМНИ

- 01 Без `timeout` – зависающие requests.
- 02 Spinner forever без cancel.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое `stale-while-revalidate`?
- 02 Когда показать cancel?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/uses-passive-event-listeners>

СДАЛ ЛИ ЗАЧЁТ

ПРОЙДИ ВСЛУХ БЕЗ ПОДГЛЯДЫВАНИЙ ЕСЛИ ЗАСТРЯЛ – ВЕРНИСЬ В НУЖНЫЙ БИЛЕТ

- ☐ [] Описать REST vs GraphQL и over/under-fetch (Q1-Q5)
- ☐ [] Описать GraphQL types/errors/cache (Q6, Q7, Q8)
- ☐ [] Описать pagination/optimistic/client design (Q9-Q12)
- ☐ [] Описать cancellation/retry/idempotency (Q13-Q16)
- ☐ [] Описать rate-limit/401-403/token refresh (Q17-Q19)
- ☐ [] Описать waterfall/prefetch/dedup/BFF (Q20-Q25)
- ☐ [] Описать contract testing/backwards compat/migration/partial/slow (Q26-Q30)